JAVA REFRESHER SERIES

Chapter 9: Annotations, Reflection and Generated Code

Depth 9/10 • Java 17+ • Practical focus • Allman-style braces

Learn to explain Java behavior that is not visible as an ordinary handwritten method: annotations, runtime reflection, records, Lombok, annotation processors and framework proxies.

How to study

Study Parts A, B and C sequentially. Read each rule, predict the output without looking, trace the example, and then run the companion Java program in IntelliJ. Examples that use Spring, Lombok or MapStruct are labeled conceptual because those require external dependencies.

Companion project: five compiled, self-contained Java 17 demonstrations with captured expected output. Other excerpts in the book demonstrate API usage and may require enclosing declarations.

JAVA CHAPTER 9 — ANNOTATIONS: METADATA, RETENTION & COMMON USES

Declarative metadata • Guide tools and frameworks • Reduce boilerplate

Smarter Code with Annotations

1 WHY IT MATTERS

Small Code. Big Impact.

Add Metadata
Annotations add metadata without changing business logic.

Used Widely
Heavily used in frameworks, validation, serialization, testing and dependency injection.

Improve Development
They improve readability and automation.

2 MENTAL MODEL

Annotations describe intent, not behavior.

Source code element → Annotation attached → Compiler / tools / framework reads it → Behavior or validation changes

- Class, method, field, etc.
- e.g. @Override or custom
- During compilation or at runtime
- Generates warnings, applies configuration, enables features, etc.

- **Annotations describe intent.**
- **They usually do not execute code by themselves.**

3 BUILT-IN ANNOTATIONS

Useful out of the box.

Annotation	Meaning
@Override	Indicates a method overrides a superclass method (compiler checks).
@Deprecated	Marks an element as deprecated (shows a warning when used).
@SuppressWarnings	Suppresses specific compiler warnings.
@FunctionalInterface	Marks that an interface is a functional interface (one abstract method).
@SafeVarargs	Asserts that a method with varargs does not perform unsafe operations.

4 CUSTOM ANNOTATION SYNTAX

Define your own annotations.

```java
import java.lang.annotation.*;

@Target({ElementType.METHOD, ElementType.TYPE})
@Retention(RetentionPolicy.RUNTIME)
public @interface Audit {
   String value() default "";  // required element
   int level() default 1;      // element with default
}
```

Usage:

```java
@Audit(value = "login",
       levell = 2)
public void doLogin() {
    ....
}
```

- Annotation elements can have **default values.**
- Use meta-annotations to control where and how it can be used.

5 META-ANNOTATIONS

Define behavior for annotations.

Meta-Annotation	Purpose
@Target	Specifies where the annotation can be used (e.g. type, method, field).
@Retention	Specifies how long the annotation is retained (SOURCE, CLASS, RUNTIME).
@Documented	Includes the annotation in Javadoc documentation.
@Inherited	Allows annotations on a class to be inherited by subclasses.
@Repeatable	Allows multiple instances of the same annotation on a single element.

6 RETENTION LEVELS

Controls when annotations are available.

Retention Policy	Where It's Available	Key Points
SOURCE	Visible in source code only (ignored by compiler).	Not stored in class file. Not available at runtime.
CLASS	Stored in the .class file (by compiler).	Available to tools during build-time. Not available through reflection at runtime.
RUNTIME	Available through reflection at runtime.	Required if frameworks or code need to read the annotation at runtime.

To read annotations using reflection at runtime, the annotation must have `@Retention(RetentionPolicy.RUNTIME)`.

7 COMMON REAL-WORLD USES

Power many Java frameworks.

Validation	e.g. @NotNull, @Size (Bean Validation).
Dependency Injection	e.g. @Inject, @Autowired (Spring).
Mapping / Serialization	e.g. @JsonProperty, @Entity (Jackson, JPA).
Test Methods	e.g. @Test, @BeforeEach (JUnit).
Configuration	e.g. @Configuration, @Bean (Spring).

8 COMMON MISTAKES

Avoid common pitfalls.

- Assuming annotations run by themselves.
- Forgetting the retention policy.
- Using the wrong target (e.g. method vs. class).
- Overusing @SuppressWarnings.
- Mixing compile-time and runtime use cases.

9 QUICK RULES / AT A GLANCE

Key takeaways to remember.

1. Use annotations to add metadata, not business logic.
2. Choose the correct retention policy.
3. Set the appropriate target.
4. Prefer built-in annotations when available.
5. Use runtime retention only when you need reflection.

10 JAVA EXAMPLE

Reflection can read annotations.

Annotation definition and usage:

```java
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface LogExecution {
    String value() default "";
}

public class Service {
    @LogExecution("calculate")
    public int calculate(int a, int b) {
        return a + b;
    }
}
```

Result (using reflection):

```java
Method m = Service.class
    .getMethod("calculate",
        int.class, int.class);
LogExecution log =
    m.getAnnotation(
        LogExecution.class);
System.out.println(
    log.value()); // "calculate"
```

PART A — ANNOTATIONS

How metadata is declared, retained, discovered and interpreted.

Lesson 01 — Annotation as metadata

CORE RULE

> An annotation records structured information about a Java declaration; it does not independently execute behavior.

WHY IT MATTERS

Think of @Override as a request for a compiler check, not an instruction to execute a superclass method. Spring and JUnit attach meanings to their own annotations by running tools that inspect metadata.

JAVA EXAMPLE — ALLMAN STYLE

```
@Override
public String toString()
{
    return "ready";
}
```

EXPECTED OUTPUT / BEHAVIOR

> No console output; the compiler validates the override.

EXECUTION TRACE

The parser sees the annotation; the compiler checks the overridden signature; only calling toString() executes code.

COMMON PITFALL

Do not assume an unknown annotation causes runtime behavior without discovering its processor or framework consumer.

APPLY THE RULE

What if the method name is toStrng()?

Answer: The compiler reports an error because no overriding method exists.

Lesson 01 — Annotations are descriptive metadata

CONCEPTUAL EXECUTION MODEL

Source declaration → attached annotation → compiler or runtime consumer → behavior. The annotation declaration itself does not invoke application logic.

DEEPER EXPLANATION / WHY IT WORKS

A custom marker only changes an application when a scanner, framework or processor is programmed to read it. @Override is interpreted by the compiler; a custom @Audit annotation can be interpreted by your application. Distinguish the declaration site from the consumer, and ask what actually performs the action.

JAVA EXAMPLE / ALLMAN BRACES

```
@Audit("payments")
public void save()
{
    // Business operation only.
}
```

RESULT / BEHAVIOR

Without an annotation consumer, this method does not automatically log anything.

COMMON PITFALL

Assuming metadata automatically implements validation, security, transactions, or logging.

APPLY THE RULE — QUESTION & ANSWER

Question: Who executes the audit action? Answer: An explicit interceptor/scanner or framework component; not @Audit by itself.

REAL-WORLD RELEVANCE

When reading Spring controllers or repositories, find the framework mechanism responsible for acting on each annotation.

Lesson 02 — Built-in annotations

CORE RULE

@Override, @Deprecated, @SuppressWarnings, @FunctionalInterface and @SafeVarargs target different programming concerns.

WHY IT MATTERS

@Deprecated is a warning signal and does not remove an API; @SuppressWarnings is scoped to where used and must not conceal real defects. @SafeVarargs is a promise about safe handling of a generic varargs parameter.

JAVA EXAMPLE — ALLMAN STYLE

```
@FunctionalInterface
interface Task
{
    void execute();
}
Task task = () -> System.out.println("run");
task.execute();
```

EXPECTED OUTPUT / BEHAVIOR

```
run
```

EXECUTION TRACE

The annotation requests a SAM check; target typing maps the lambda to execute(); the invocation prints run.

COMMON PITFALL

@FunctionalInterface permits default/static methods but requires one abstract method, ignoring Object-equivalent signatures.

APPLY THE RULE

Can an interface have two default methods and remain functional?

Answer: Yes, provided it has exactly one abstract method.

Lesson 02 — Built-in annotations and compiler checks

CONCEPTUAL EXECUTION MODEL

@Override → compiler verifies a valid override; @Deprecated → usage warnings; @FunctionalInterface → SAM-contract check.

DEEPER EXPLANATION / WHY IT WORKS

The compiler can reject a signature mismatch annotated with @Override. @Deprecated can include since and forRemoval metadata. @SuppressWarnings is scoped: prefer the smallest expression, statement or method that truly needs suppression. @SafeVarargs is not a general license to ignore unsafe generic-array usage.

JAVA EXAMPLE / ALLMAN BRACES

```
class Parent
{
    void run() {}
}
class Child extends Parent
{
    @Override
    void run() {}
}
```

RESULT / BEHAVIOR

Compiles; changing Child.run to run(int) causes an @Override compilation error.

COMMON PITFALL

Suppressing unchecked warnings without verifying the cast or heap-pollution risk.

APPLY THE RULE — QUESTION & ANSWER

Question: Does @FunctionalInterface allow default methods? Answer: Yes, if exactly one abstract method remains.

REAL-WORLD RELEVANCE

Treat compiler annotations as safety checks, not decoration.

Lesson 03 — Define a custom annotation

CORE RULE

An annotation type is declared with @interface and its elements are no-argument methods returning permitted element types.

WHY IT MATTERS

Allowed element types include primitives, String, Class, enums, annotations and one-dimensional arrays of these. Defaults are constant values; annotation instances are not conventional mutable objects.

JAVA EXAMPLE — ALLMAN STYLE

```
public @interface Audit
{
    String value();
    int priority() default 1;
}
@Audit(value = "login", priority = 2)
void signIn()
{
}
```

EXPECTED OUTPUT / BEHAVIOR

No output; source declares and uses metadata.

EXECUTION TRACE

The element named value is supplied; priority overrides its default; a consumer must inspect Audit to give it meaning.

COMMON PITFALL

You cannot use arbitrary domain objects or null as annotation element values.

APPLY THE RULE

What does @Audit("logout") mean?

Answer: It assigns value="logout" and uses priority=1.

Lesson 03 — Custom annotation elements and defaults

CONCEPTUAL EXECUTION MODEL

@interface defines an annotation type; its elements are parameterless members; defaults let callers omit optional values.

DEEPER EXPLANATION / WHY IT WORKS

Annotation element types are restricted to primitives, String, Class, enums, annotation types, and one-dimensional arrays thereof. Values must be compile-time constants (or class literals / valid annotation values). The specially named value() element supports shorthand when it is the only element supplied.

JAVA EXAMPLE / ALLMAN BRACES

```
public @interface Route
{
    String value();
    int priority() default 0;
}
@Route("/status")
class StatusHandler {}
```

RESULT / BEHAVIOR

Route.value = /status; Route.priority = 0.

COMMON PITFALL

Trying to use arbitrary POJOs, lists or runtime-calculated values as annotation attributes.

APPLY THE RULE — QUESTION & ANSWER

Question: Can an annotation member be null? Answer: No; annotations cannot use null as an element value.

REAL-WORLD RELEVANCE

Common Spring-style annotations pass configuration constants to container scanners.

Lesson 04 — Targets and placement

CORE RULE

@Target defines the kinds of Java elements that may legally carry an annotation.

WHY IT MATTERS

METHOD, FIELD, TYPE, PARAMETER, CONSTRUCTOR and TYPE_USE are different placement categories. TYPE_USE covers annotations on type occurrences and is not interchangeable with the ordinary TYPE declaration target.

JAVA EXAMPLE — ALLMAN STYLE

```
import java.lang.annotation.*;
@Target({ElementType.METHOD, ElementType.TYPE})
@interface Marker
{
}
@Marker
class Service
{
    @Marker
    void run()
    {
    }
}
```

EXPECTED OUTPUT / BEHAVIOR

No output; class and method use are legal.

EXECUTION TRACE

The compiler checks the annotation targets; illegal field placement fails compilation; legal placement remains only as retention permits.

COMMON PITFALL

Do not conflate ElementType.TYPE with type-use annotation placement.

APPLY THE RULE

Would @Marker on a field compile?

Answer: No: FIELD is absent from @Target.

Lesson 04 — Targets and type-use annotations

CONCEPTUAL EXECUTION MODEL

@Target constrains legal declaration sites; TYPE_USE is different from TYPE and can apply to uses of a type.

DEEPER EXPLANATION / WHY IT WORKS

ElementType.METHOD, FIELD, PARAMETER, CONSTRUCTOR, TYPE and TYPE_USE are not interchangeable. TYPE_USE is valuable for nullness annotations and static analysis, while TYPE targets declarations such as classes. Repeating an annotation on an unsupported element fails at compile time.

JAVA EXAMPLE / ALLMAN BRACES

```
@Target(ElementType.METHOD)
@interface Timed {}

class Handler
{
    @Timed
    void run() {}
}
```

RESULT / BEHAVIOR

Compiles because Timed is attached to a method.

COMMON PITFALL

Assuming a method-targeted annotation can be placed on its return type or on a field.

APPLY THE RULE — QUESTION & ANSWER

Question: Which target supports @Nullable String as a type use? Answer: TYPE_USE.

REAL-WORLD RELEVANCE

Understand annotation targeting when building validators and static checks.

Lesson 05 — Retention policies

CORE RULE

SOURCE annotations disappear during compilation, CLASS annotations remain in class files, and RUNTIME annotations are readable through ordinary reflection.

WHY IT MATTERS

SOURCE can support compile-time checks, CLASS supports bytecode tooling, and RUNTIME supports framework discovery. Retention describes visibility, not whether a processor is automatically run.

JAVA EXAMPLE — ALLMAN STYLE

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface Flag
{
}
class Job
{
    @Flag
    public void run()
    {
    }
}
```

EXPECTED OUTPUT / BEHAVIOR

No output until a reflective lookup is executed.

EXECUTION TRACE

Compilation preserves Flag in the class file; runtime reflection can discover it; a caller still must check and act.

COMMON PITFALL

Default retention is CLASS, not RUNTIME.

APPLY THE RULE

Will getAnnotation() find Flag without @Retention?

Answer: No, not through ordinary runtime reflection.

Lesson 05 — Retention and lifecycle boundaries

CONCEPTUAL EXECUTION MODEL

SOURCE → source processing only; CLASS → retained in class file; RUNTIME → accessible to reflection.

DEEPER EXPLANATION / WHY IT WORKS

Retention controls the visibility contract, not whether a consumer is automatically created. A compile-time annotation processor can inspect SOURCE or CLASS retention declarations while compiling source. Runtime reflection cannot discover a CLASS-only annotation with getAnnotation(). RUNTIME should be intentional because it contributes runtime metadata.

JAVA EXAMPLE / ALLMAN BRACES

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface Endpoint {}
```

RESULT / BEHAVIOR

getDeclaredMethod(...).isAnnotationPresent(Endpoint.class) can return true.

COMMON PITFALL

Expecting reflection to see an annotation with CLASS or SOURCE retention.

APPLY THE RULE — QUESTION & ANSWER

Question: Does SOURCE mean an annotation processor cannot read it? Answer: No; processors inspect source elements during compilation.

REAL-WORLD RELEVANCE

Choose retention based on the actual consumer: compiler tool, bytecode tool, or runtime framework.

Lesson 06 — Inherited, repeatable and documented

CORE RULE

@Inherited affects only class-level annotations and reflective lookup through superclass chains; @Repeatable allows repeated instances.

WHY IT MATTERS

@Inherited does not propagate annotations from interfaces or from overridden methods. @Documented makes annotations appear in generated documentation, not executable behavior.

JAVA EXAMPLE — ALLMAN STYLE

```
@Inherited
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
@interface Tag
{
}
@Tag
class Parent
{
}
class Child extends Parent
{
}
```

EXPECTED OUTPUT / BEHAVIOR

Child.class.getAnnotation(Tag.class) != null → true

EXECUTION TRACE

Child has no direct Tag; runtime lookup follows inherited class-level metadata; declared-annotation lookup remains false.

COMMON PITFALL

Do not mistake annotation inheritance for Java method inheritance.

APPLY THE RULE

Lesson 06 — Inherited, repeatable and documented metadata

CONCEPTUAL EXECUTION MODEL

@Inherited applies to class-level inheritance via Class API; @Repeatable supports multiple annotations on one element.

DEEPER EXPLANATION / WHY IT WORKS

@Inherited does not copy annotations onto overridden methods and does not propagate from interfaces to implementing classes. With @Repeatable, the compiler may wrap multiple annotation instances in a container annotation; getAnnotationsByType() supports repeated declarations. @Documented only controls Javadoc inclusion.

JAVA EXAMPLE / ALLMAN BRACES

```
@Inherited
@Retention(RetentionPolicy.RUNTIME)
@interface Role {}

@Role class Parent {}
class Child extends Parent {}
```

RESULT / BEHAVIOR

Child.class.isAnnotationPresent(Role.class) is true.

COMMON PITFALL

Assuming @Inherited works on interface implementations or inherited methods.

APPLY THE RULE — QUESTION & ANSWER

Question: Which API reads repeated annotations? Answer: getAnnotationsByType(TheAnnotation.class).

REAL-WORLD RELEVANCE

Important when diagnosing why a framework sees or does not see inherited configuration.

Does @Inherited on a method annotation work?

Answer: No; it is effective for class declarations only.

Lesson 07 — Compile-time processing versus runtime consumers

CORE RULE

An annotation processor runs during compilation, while a runtime framework can inspect RUNTIME annotations while the application executes.

WHY IT MATTERS

Compile-time processors commonly produce source files or diagnostics. Runtime consumers commonly register handlers, route requests, or apply validation. The annotation is a marker that another mechanism interprets.

JAVA EXAMPLE — ALLMAN STYLE

```
@Retention(RetentionPolicy.RUNTIME)
@interface Route
{
    String path();
}
@Route(path = "/orders")
class OrderHandler
{
}
```

EXPECTED OUTPUT / BEHAVIOR

No route is registered by ordinary Java alone.

EXECUTION TRACE

Compile the class; metadata survives; a framework scans it and may register a path if configured to do so.

COMMON PITFALL

Do not confuse annotation retention with compilation code generation.

APPLY THE RULE

Does declaring @Route automatically create an HTTP endpoint?

Answer: No. A runtime framework must be configured to interpret it.

Lesson 07 — Compile-time processors versus runtime scanning

CONCEPTUAL EXECUTION MODEL

javac annotation-processing rounds → processor discovers Element metadata → filer writes a new source → future compilation compiles it.

DEEPER EXPLANATION / WHY IT WORKS

A javax.annotation.processing.AbstractProcessor (in javax, even for Java 17) uses RoundEnvironment, TypeElement, Filer and Messager. Register it through META-INF/services/javax.annotation.processing.Processor or explicitly with -processor. In a single-module demo, compile the processor first, then compile the annotated consumer with -processorpath. Runtime reflection is unrelated to source generation.

JAVA EXAMPLE / ALLMAN BRACES

```
@SupportedAnnotationTypes("demo.GenerateGreeting")
@SupportedSourceVersion(SourceVersion.RELEASE_17)
public class Generator extends AbstractProcessor
{
    public boolean process(Set<? extends TypeElement> a, RoundEnvironment r)
    {
        // Discover elements and write source via Filer.
        return true;
    }
}
```

RESULT / BEHAVIOR

A source file appears under the configured generated-sources directory during javac.

COMMON PITFALL

Expecting javac to discover a processor when its JAR is not on the processor path.

APPLY THE RULE — QUESTION & ANSWER

Question: Is runtime retention required to generate Java source during compilation? Answer: No.

REAL-WORLD RELEVANCE

The companion project includes a real two-stage processor compilation script.

Lesson 08 — Find runtime annotations

CORE RULE

Use isAnnotationPresent() and getAnnotation() on Class, Method, Field or Constructor objects when annotation retention is RUNTIME.

WHY IT MATTERS

Reflection can inspect class-level declarations and executable members. AnnotatedElement has different declared versus inherited lookup methods. Robust tools handle missing annotations rather than dereferencing null.

JAVA EXAMPLE — ALLMAN STYLE

```
Method m = Handler.class.getDeclaredMethod("run");
Action a = m.getAnnotation(Action.class);
System.out.println(a.value());
```

EXPECTED OUTPUT / BEHAVIOR

audit

EXECUTION TRACE

Find the method by exact signature; retrieve the runtime annotation; read its element through the generated annotation proxy.

COMMON PITFALL

getDeclaredMethod requires exact parameter types; overloaded names alone are insufficient.

APPLY THE RULE

What if an annotation is missing?

Answer: getAnnotation returns null; handle it before reading elements.

Lesson 08 — Reading annotations accurately

CONCEPTUAL EXECUTION MODEL

getAnnotation on AnnotatedElement → annotation proxy with element values; getDeclaredAnnotation excludes inherited declarations.

DEEPER EXPLANATION / WHY IT WORKS

Class, Method, Field and Constructor implement AnnotatedElement. Use declared variants when inheritance matters, and be explicit about exact parameter types in getDeclaredMethod. Reading an annotation safely requires checking for null or using isAnnotationPresent. The values returned are annotation-interface methods rather than JavaBean getter properties.

JAVA EXAMPLE / ALLMAN BRACES

```
Method m = Service.class.getDeclaredMethod("run");
Audit a = m.getAnnotation(Audit.class);
if (a != null)
{
    System.out.println(a.value());
}
```

RESULT / BEHAVIOR

Prints the Audit value only when the annotation exists at runtime.

COMMON PITFALL

Dereferencing a null annotation or requesting the wrong overloaded method signature.

APPLY THE RULE — QUESTION & ANSWER

Question: Will getDeclaredAnnotation inherit a superclass annotation? Answer: No.

REAL-WORLD RELEVANCE

A common mechanism behind routing, validation and method-level interceptors.

JAVA CHAPTER 9 — REFLECTION: INSPECTING CLASSES, METHODS, FIELDS & CONSTRUCTORS

Discover type information at runtime • Invoke dynamically • Use carefully

1 WHY IT MATTERS

- Used by frameworks, libraries and containers (e.g. dependency injection).
- Enables serialization, testing tools, ORMs and plugin systems.
- Lets you inspect types, methods, fields and annotations at runtime.
- Provides flexibility for frameworks that work with unknown types.

2 MENTAL MODEL

Class object	Inspect members	Read metadata / annotations	Create object or invoke	Framework behavior
Get a Class<?> from a type.	Find fields, methods, constructors, etc.	Get names, types modifiers and annotations.	Construct instances and call methods or access fields.	Use the information to provide dynamic features.

- Reflection lets you examine and use class information at runtime.
- It is often used by frameworks and libraries, not usually in everyday application code.

3 GETTING A CLASS OBJECT

`MyType.class`	Compile-time reference. Use when you know the type at compile time.
`obj.getClass()`	From an existing instance. Use when you have an object and want its runtime type.
`Class.forName("com.example.MyType")`	By fully qualified name. Use when the class name is known only at runtime (may throw ClassNotFoundException).

4 CORE APIS

API	Purpose
`Class<?>`	Represents a class or interface at runtime. Entry point for inspecting its members.
`Field`	Represents a field (variable). Read or modify values.
`Method`	Represents a method. Invoke it dynamically.
`Constructor<?>`	Represents a constructor. Create new instances.
`Modifier`	Utility to check modifiers (public, private, static, etc).
`Package`	Represents package information for a class.
`Annotation`	Read annotations present on classes, methods, fields, etc.

5 COMMON REFLECTION TASKS

- **Inspect method names** — Get all declared or public methods and their signatures.
- **List fields** — Get fields, their types, modifiers and current values.
- **Read annotations** — Check for annotations on classes, methods or fields.
- **Call a constructor** — Find a constructor and create a new instance.
- **Invoke a method** — Call a method with arguments and get the return value.
- **Read or set a field** — Get or update the value of a field (even private with access).

6 JAVA EXAMPLE

```java
import java.lang.reflect.*;
class Calculator {
    private int calculate(int a, int b) {
        return a + b;
    }
}
public class ReflectionDemo {
    public static void main(String[] args) throws Exception {
        // 1. Get Class object
        Class<?> clazz = Calculator.class;
        // 2. Get declared method with parameter types
        Method method = clazz.getDeclaredMethod(
            "calculate", int.class, int.class);
        // 3. Create instance using constructor
        Object obj = clazz.getDeclaredConstructor().newInstance();
        // 4. Allow access to private method and invoke it
        method.setAccessible(true);
        Object result = method.invoke(obj, 3, 4);
    }   System.out.println("Result: " + result);
}
```

Output (console)

```
Result: 7
```

7 ACCESS CONTROL & setAccessible

- Reflection can access private, protected and package-private members using `setAccessible(true)`.
- Useful for frameworks and testing tools.
- Should be used sparingly, as it breaks encapsulation.
- May be restricted in modern Java (module system, security manager, or runtime policies).

8 COMMON MISTAKES

- Using reflection when normal code would do.
- Ignoring checked exceptions from reflection APIs.
- Passing wrong parameter types to methods/constructors.
- Forgetting that reflection is slower and less type-safe.
- Breaking encapsulation or modifying private state.
- Not handling `NoSuchMethodException`, `IllegalAccessException` or `InvocationTargetException`.

9 PERFORMANCE & SAFETY

- Reflection is slower than direct method calls.
- May be restricted by the module system, security manager or runtime policies.
- Cache reflective lookups (Class, Method, Field) if used repeatedly.
- Validate input and handle exceptions properly.
- Use reflection with caution and only when needed.

10 QUICK RULES / AT A GLANCE

1. Prefer normal code first; use reflection only when needed.
2. Use reflection at boundaries (frameworks, libraries, plugins).
3. Cache Class, Method and Field objects for repeated use.
4. Handle all reflection exceptions and provide clear messages.
5. Respect encapsulation and avoid changing private state unless necessary.
6. Consider module restrictions and security policies.
7. Keep reflection code small, well-documented and test it.

PART B — REFLECTION

How to inspect Java types dynamically and understand framework reflection.

Lesson 09 — Class objects and class loading

CORE RULE

Foo.class, obj.getClass() and Class.forName(name) all provide Class objects through different routes.

WHY IT MATTERS

A class literal is available when the type is known; getClass() reveals an object's actual runtime class; Class.forName is useful for name-based discovery and can initialize a class with its usual overload.

JAVA EXAMPLE — ALLMAN STYLE

```
Class<String> type = String.class;
System.out.println(type.getSimpleName());
System.out.println("hello".getClass() == type);
```

EXPECTED OUTPUT / BEHAVIOR

```
String
true
```

EXECUTION TRACE

The literal represents String; the instance runtime type is also String; identity comparison matches the same Class object.

COMMON PITFALL

Class.forName can raise ClassNotFoundException; do not invent class names from untrusted input.

APPLY THE RULE

Does a subclass object return the superclass in getClass()?

Answer: No; getClass() reports the actual runtime class.

Lesson 09 — Class metadata and class loading

CONCEPTUAL EXECUTION MODEL

Type.class / obj.getClass() / Class.forName(name) → Class descriptor. Initialization is a separate concern.

DEEPER EXPLANATION / WHY IT WORKS

The Class object describes a loaded type, not an instance of that class. The literal Type.class does not itself construct a Type object. Class.forName(String) typically loads and initializes the named class; overloads can control initialization. Consider class loaders and package-qualified names rather than assuming a single global class namespace.

JAVA EXAMPLE / ALLMAN BRACES

```
Class<?> a = String.class;
Class<?> b = "x".getClass();
System.out.println(a == b);
```

RESULT / BEHAVIOR

true

COMMON PITFALL

Confusing Class metadata objects with actual T instances.

APPLY THE RULE — QUESTION & ANSWER

Question: Can two class loaders define separate Class objects for the same binary class name? Answer: Yes.

REAL-WORLD RELEVANCE

Framework scanning and plugin systems rely on type metadata and class loading.

Lesson 10 — getMethods versus getDeclaredMethods

CORE RULE

getMethods() finds public inherited and declared methods; getDeclaredMethods() finds every method declared directly on that class, including private ones.

WHY IT MATTERS

This is a frequent point of confusion when inspecting libraries. Declared member discovery and access permission are separate: finding a private member does not mean you can legally invoke it.

JAVA EXAMPLE — ALLMAN STYLE

```
Method[] publicMethods = String.class.getMethods();
Method[] declaredMethods = String.class.getDeclaredMethods();
System.out.println(publicMethods.length > 0);
System.out.println(declaredMethods.length > 0);
```

EXPECTED OUTPUT / BEHAVIOR

```
true
true
```

EXECUTION TRACE

One API traverses public inheritance; the other inspects directly declared methods; results can overlap but are not equivalent.

COMMON PITFALL

Never rely on unspecified reflection array order.

APPLY THE RULE

How would you retrieve a private method declared locally?

Answer: Use getDeclaredMethod; invoking may still require access permission.

Lesson 10 — Inherited versus declared method discovery

CONCEPTUAL EXECUTION MODEL

getMethods → public inherited + public declared; getDeclaredMethods → methods declared on that class, regardless of modifier.

DEEPER EXPLANATION / WHY IT WORKS

Neither API guarantees useful ordering. getMethod(String, Class...) searches public members, while getDeclaredMethod searches only the declaring type. Watch for bridge and synthetic methods generated by Java compilers when generics and overrides are involved. Sort names explicitly for deterministic diagnostic output.

JAVA EXAMPLE / ALLMAN BRACES

```
for (Method m : Child.class.getDeclaredMethods())
{
    System.out.println(m.getName());
}
```

RESULT / BEHAVIOR

Prints only methods declared by Child; order is unspecified.

COMMON PITFALL

Assuming getDeclaredMethods is the same as getMethods, or relying on returned iteration order.

APPLY THE RULE — QUESTION & ANSWER

Question: Which API can find a private Child method? Answer: getDeclaredMethod / getDeclaredMethods.

REAL-WORLD RELEVANCE

Needed for metadata scanners and diagnostics of generated bridge methods.

Lesson 11 — Reflective invocation and exceptions

CORE RULE

Method.invoke(receiver, args) can invoke a method dynamically, returning Object for non-void methods.

WHY IT MATTERS

You must supply the exact method signature and receiver. InvocationTargetException wraps the exception thrown by the invoked target. For a static method, the receiver may be null.

JAVA EXAMPLE — ALLMAN STYLE

```
Method add = Calculator.class.getMethod(
    "add", int.class, int.class);
Object result = add.invoke(new Calculator(), 3, 4);
System.out.println(result);
```

EXPECTED OUTPUT / BEHAVIOR

```
7
```

EXECUTION TRACE

Find public add(int,int); instantiate Calculator; invoke with boxed arguments; Java boxes the int return value into Object.

COMMON PITFALL

Inspect InvocationTargetException.getCause() before reporting a framework failure.

APPLY THE RULE

Why is the return value Object?

Answer: The reflection API abstracts over all possible method result types.

Lesson 11 — Invocation and exception unwrapping

CONCEPTUAL EXECUTION MODEL

lookup Method → verify receiver and arguments → invoke → target throws? InvocationTargetException wraps the underlying failure.

DEEPER EXPLANATION / WHY IT WORKS

Method.invoke takes an instance for nonstatic methods and null for static methods. Wrong parameter types lead to IllegalArgumentException; inaccessible methods lead to IllegalAccessException or module-access failures. When the called method itself throws, inspect InvocationTargetException.getCause() so the diagnostic preserves the real failure.

JAVA EXAMPLE / ALLMAN BRACES

```
try
{
    method.invoke(target, args);
}
catch (InvocationTargetException ex)
{
    throw new IllegalStateException("Handler failed", ex.getCause());
}
```

RESULT / BEHAVIOR

The original handler exception is retained as the new exception cause.

COMMON PITFALL

Logging only InvocationTargetException and losing the underlying failure cause.

APPLY THE RULE — QUESTION & ANSWER

Question: Which exception wraps an exception thrown by an invoked method? Answer: InvocationTargetException.

REAL-WORLD RELEVANCE

Distinguish framework infrastructure failures from failures in actual business methods.

Lesson 12 — Constructors and dynamic instantiation

CORE RULE

Constructor<T>.newInstance creates objects through a chosen constructor; getConstructor finds a public constructor.

WHY IT MATTERS

Prefer type-safe ordinary constructors when the type is known. Reflection is useful for framework assembly, plugin loading, test tooling and serialization, but exposes more failure paths.

JAVA EXAMPLE — ALLMAN STYLE

```
Constructor<Calculator> c =
    Calculator.class.getConstructor();
Calculator value = c.newInstance();
System.out.println(value.add(3, 4));
```

EXPECTED OUTPUT / BEHAVIOR

```
7
```

EXECUTION TRACE

Locate the no-arg constructor; invoke it; obtain an instance; call a normal public method on the instance.

COMMON PITFALL

Class.newInstance() is deprecated; use a Constructor object.

APPLY THE RULE

What if only a private constructor exists?

Answer: getConstructor fails; getDeclaredConstructor can discover it, but access may be denied.

Lesson 12 — Constructors and instantiated type contracts

CONCEPTUAL EXECUTION MODEL

getDeclaredConstructor(parameterTypes) → Constructor → newInstance(arguments); validation still runs.

DEEPER EXPLANATION / WHY IT WORKS

Reflection does not bypass normal constructor logic. A constructor throwing an exception produces an InvocationTargetException, with the original cause available. Constructors have exact signatures; primitive int and wrapper Integer differ at lookup. Prefer explicit constructors and normal new for your own application code.

JAVA EXAMPLE / ALLMAN BRACES

```
Constructor<User> ctor = User.class.getDeclaredConstructor(String.class);
User user = ctor.newInstance("Ada");
```

RESULT / BEHAVIOR

Creates User("Ada") if the constructor is accessible.

COMMON PITFALL

Requesting Integer.class when the declared constructor parameter is int.class.

APPLY THE RULE — QUESTION & ANSWER

Question: How does reflection report a failure thrown by the constructor body? Answer: InvocationTargetException.

REAL-WORLD RELEVANCE

Object creation by serializers or dependency injection containers can use constructor metadata.

Lesson 13 — Fields, modifiers and generic signatures

CORE RULE

Field, Method and Constructor expose names, modifiers, parameter types, and generic type information.

WHY IT MATTERS

Use Modifier.isPublic and related helpers to inspect visibility. getType() returns a raw Class; getGenericType() retains reflective generic declarations that may matter to dependency injection or serializers.

JAVA EXAMPLE — ALLMAN STYLE

```
Field field = User.class.getDeclaredField("name");
System.out.println(field.getName());
System.out.println(field.getType().getSimpleName());
```

EXPECTED OUTPUT / BEHAVIOR

```
name
String
```

EXECUTION TRACE

A Field object models the declaration; inspecting its type does not read any object’s field value.

COMMON PITFALL

Accessing a private field through setAccessible can violate module boundaries and encapsulation.

APPLY THE RULE

Does getDeclaredField retrieve inherited fields?

Answer: No. It checks only the declaring class.

Lesson 13 — Fields, modifiers and generic signatures

CONCEPTUAL EXECUTION MODEL

Field metadata exposes name, declared type, modifiers and generic Type; declared generic type may be parameterized.

DEEPER EXPLANATION / WHY IT WORKS

getType() returns the raw Class, while getGenericType() may expose ParameterizedType like List. Modifier.isStatic and Modifier.isFinal are for structural inspection. Field.get/set is an access-sensitive operation: introspection is easier and safer than mutating private state. Use ordinary getters or constructors whenever possible.

JAVA EXAMPLE / ALLMAN BRACES

```
Field f = User.class.getDeclaredField("tags");
System.out.println(f.getType());
System.out.println(f.getGenericType());
```

RESULT / BEHAVIOR

Possible result: interface java.util.List and java.util.List.

COMMON PITFALL

Treating a field with generic type List as though getType() returns List.class.

APPLY THE RULE — QUESTION & ANSWER

Question: Which reflection method preserves generic signature detail? Answer: getGenericType().

REAL-WORLD RELEVANCE

Libraries inspect declared DTO fields and their nested types for serialization.

Lesson 14 — Access checks and JPMS boundaries

CORE RULE

setAccessible(true) may suppress Java language access checks, but Java module boundaries can still deny deep reflection.

WHY IT MATTERS

Frameworks often use module opens clauses where necessary. Prefer public APIs, injectable constructors, and well-supported framework integration over forcing private access. InaccessibleObjectException is a common symptom of strong encapsulation.

JAVA EXAMPLE — ALLMAN STYLE

```
try
{
    field.setAccessible(true);
}
catch (RuntimeException ex)
{
    System.out.println("access denied");
}
```

EXPECTED OUTPUT / BEHAVIOR

Output depends on module access configuration.

EXECUTION TRACE

The VM checks whether deep reflection is permitted; access may succeed within an open package or fail across strongly encapsulated modules.

COMMON PITFALL

Never claim setAccessible always works on modern Java.

APPLY THE RULE

Which module directive permits reflective access to a package?

Answer: opens, or an opens-to-module directive for targeted access.

Lesson 14 — Module boundaries and accessible members

CONCEPTUAL EXECUTION MODEL

AccessibleObject.trySetAccessible() may fail for inaccessible module boundaries even when reflective metadata is visible.

DEEPER EXPLANATION / WHY IT WORKS

The Java Platform Module System separates exports (ordinary public access) from opens (deep reflective access). Since modern Java, setAccessible(true) cannot always cross a strongly encapsulated module boundary. Prefer public APIs and framework-supported --add-opens only as a measured workaround; do not teach global private-field override as normal business code.

JAVA EXAMPLE / ALLMAN BRACES

```
if (!field.trySetAccessible())
{
    throw new IllegalStateException("Module does not open this package");
}
```

RESULT / BEHAVIOR

The application fails with an actionable access message when access cannot be enabled.

COMMON PITFALL

Assuming setAccessible(true) always succeeds in Java 17 and later.

APPLY THE RULE — QUESTION & ANSWER

Question: Does trySetAccessible always succeed? Answer: No; module openness and Java access restrictions still apply.

REAL-WORLD RELEVANCE

Framework compatibility and plugin debugging frequently involve module opens.

Lesson 15 — Reflection cost and safe boundaries

CORE RULE

Reflection adds lookup and invocation overhead, failure modes and weaker compile-time guarantees; use it where dynamic behavior is the requirement.

WHY IT MATTERS

Cache known Method, Field and Constructor objects when repeated lookup is demonstrably costly. Validate names and input before loading or calling unknown types. Normal Java invocation is preferred for routine business logic.

JAVA EXAMPLE — ALLMAN STYLE

```
Method method = Calculator.class.getMethod(
    "add", int.class, int.class);
for (int i = 0; i < 2; i++)
{
    System.out.println(method.invoke(
        new Calculator(), i, 1));
}
```

EXPECTED OUTPUT / BEHAVIOR

```
1
2
```

EXECUTION TRACE

Resolve the method once; invoke it twice; output follows target execution as usual.

COMMON PITFALL

Do not use reflection as a replacement for polymorphism and interfaces.

APPLY THE RULE

Should every method call use reflection for flexibility?

Answer: No; ordinary calls are safer, clearer, and typically faster.

Lesson 15 — Reflection cost, caching and trust boundaries

CONCEPTUAL EXECUTION MODEL

inspect once → cache Class/Method metadata → validate input → invoke at a controlled boundary.

DEEPER EXPLANATION / WHY IT WORKS

Reflection often adds lookup, boxing and invocation overhead compared with direct calls, but premature assumptions about speed can mislead: measure realistic use cases. The larger risk is maintenance and security: dynamic strings and untrusted method names are hard to validate. Restrict what can be invoked and never reflectively execute arbitrary user-supplied class names or methods.

JAVA EXAMPLE / ALLMAN BRACES

```
Map<String, Method> handlers = Map.of(
    "status", Service.class.getDeclaredMethod("status"));
```

RESULT / BEHAVIOR

Only the named, explicitly permitted handler is available.

COMMON PITFALL

Turning public request parameters into arbitrary reflective method invocations.

APPLY THE RULE — QUESTION & ANSWER

Question: What should you cache? Answer: Lookup metadata where repeated calls make it useful.

REAL-WORLD RELEVANCE

A framework uses reflection for boundary wiring, not for every simple business operation.

JAVA CHAPTER 9 — GENERATED CODE: RECORDS, LOMBOK, PROXIES & ANNOTATION PROCESSORS

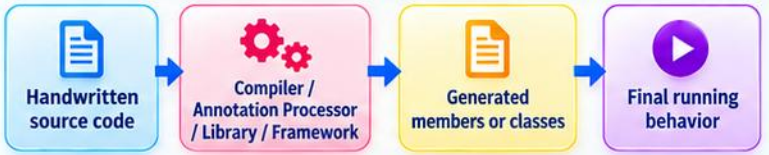
Understand code you did not write • Know what the compiler or framework creates

JAVA 17+

1 WHY IT MATTERS

- Modern Java projects often rely on generated code.
- It reduces boilerplate and speeds up development.
- Generated code can hide important behavior, making it harder to understand, debug and maintain applications.

2 MENTAL MODEL



Your classes, annotations, configuration

Processes code, analyzes annotations, applies rules

Additional code (e.g. methods, classes, proxies, metadata)

Executed by the JVM at compile time or runtime

3 COMMON SOURCES OF GENERATED CODE

Source	What it generates
Records	Constructor, accessors, equals, hashCode, toString and other methods.
Lombok	Getters, setters, constructors, builders and more via annotations.
Annotation Processors	New source files or additional members based on annotations.
Dynamic Proxies	Proxy classes that implement interfaces at runtime (e.g. java.lang.reflect.Proxy).
ORM / Framework Proxies	Subclass or proxy classes for lazy loading, transactions, security, dependency injection.
IDE-Generated Methods	Methods like getters/setters, equals/hashCode, toString, constructors and more.

4 RECORDS

Records automatically generate common members.

Java

```
public record Person(String name, int age) { }
```

Generated members include:

- Canonical constructor (Person(String name, int age))
- Accessors (name(), age())
- equals(Object)
- hashCode()
- toString() (e.g. Person[name=Ana, age=30])

Example toString() output:

```
Person[name=Ana, age=30]
```

5 LOMBOK

Lombok generates boilerplate code using annotations.

Java

```
import lombok.*;
@Data
@Builder
public class User {
    private String name;
    private int age;
}
```

Common Lombok annotations

@Getter	Generates getter methods.
@Setter	Generates setter methods.
@Builder	Builder API for object creation.
@Data	Combines getters, setters, toString, equals and hashCode.
@RequiredArgsConstructor	Constructor for final fields.

- Works via annotation processing and code generation.
- Methods are not visible in the source file but are generated.
- Requires the Lombok dependency and IDE support.

6 ANNOTATION PROCESSORS

Compile-time tools that generate source code.

Java

```
@Mapper
public interface UserMapper {
    UserDto toDto(User user);
}
```

Common use cases

Mapping code	e.g. object mappers
Serialization	e.g. JSON, XML, protocol buffers
DSL or builders	e.g. query builders
Client generators	e.g. REST or gRPC clients
Other tools	e.g. configuration, metadata

- Runs during compilation and generates `.java` source files.
- Generated sources are usually in `target/generated-sources` or `build/generated`.
- Examples: mapping libraries, serialization tools, configuration and more.

7 RUNTIME PROXIES

Frameworks may create proxy classes at runtime.

- **Transactions** — add transaction boundaries.
- **Lazy loading** — load data on demand.
- **Security** — enforce access control.
- **Logging** — add logging or monitoring.
- **Dependency injection** — manage bean lifecycles.

Proxies can implement interfaces or extend classes, and often add cross-cutting behavior at runtime.

8 HOW TO INSPECT GENERATED CODE

IDE navigation	Go to declaration, open generated sources, use IDE features to view generated members.
Decompiled class view	Use "Show bytecode" or a decompiler to view compiled classes.
Generated sources folder	Check target/generated-sources or build/generated-sources folders.
Stack traces	Generated classes often appear in stack traces. Useful for proxies or generated implementations.
Framework proxy names	Look for common naming patterns (e.g. $Proxy, $$EnhancerBySpringCGLIB$$).

9 COMMON MISTAKES

- Assuming behavior exists only in handwritten source code.
- Forgetting that equals, hashCode or toString are generated (e.g. records or Lombok).
- Not realizing proxies change the actual class type and can intercept methods.
- Debugging issues without checking generated code.
- Overusing boilerplate-hiding annotations and losing control of application behavior.

10 QUICK RULES / AT A GLANCE

- Records are great for immutable data carriers.
- Lombok is useful but requires team discipline.
- Inspect generated code when debugging or seeing unexpected behavior.
- Know the difference between compile-time and runtime generation.
- Generated code can improve productivity, but always understand what it creates.

11 JAVA EXAMPLE

Record vs Lombok-like example

Java

```
// Record (generated members)
public record Person(String name, int age) { }
// Lombok-style (conceptual)
@Data
class User {
    private String name;
    private int age;
}
```

Example output

```
System.out.println(new Person("Ana", 30));
// Output:
Person(name=Ana, age=30]
```

PART C — GENERATED CODE

How compiler-generated and runtime-generated behavior enters an application.

Lesson 16 — Records as data carriers

CORE RULE

A record defines a transparent nominal data carrier with generated accessors, constructor, equals, hashCode and toString.

WHY IT MATTERS

Records are shallowly immutable with final component fields, not deeply immutable if they contain mutable objects. They are excellent for DTO-like snapshots and small value-oriented types, but not every mutable entity.

JAVA EXAMPLE — ALLMAN STYLE

```
record Person(String name, int age)
{
}
System.out.println(new Person("Ana", 30));
```

EXPECTED OUTPUT / BEHAVIOR

```
Person[name=Ana, age=30]
```

EXECUTION TRACE

The compiler derives component fields/accessors and common value semantics; record toString formats the components.

COMMON PITFALL

Do not assume a List component becomes immutable just because a record stores it.

APPLY THE RULE

Can a record extend an arbitrary class?

Answer: No; records implicitly extend java.lang.Record.

Lesson 16 — Records are shallowly immutable carriers

CONCEPTUAL EXECUTION MODEL

record components → constructor + accessors + equals/hashCode/toString; record object fields are final but referenced objects may be mutable.

DEEPER EXPLANATION / WHY IT WORKS

A record automatically exposes its components as accessors such as name(), not getName(). Its equals and hashCode are component based. Arrays and mutable lists inside a record remain mutable unless you defensively copy them. Records cannot extend arbitrary classes, but can implement interfaces; they are useful for DTO-style values, not mutable entity lifecycles.

JAVA EXAMPLE / ALLMAN BRACES

```
record Order(String id, List<String> lines)
{
    Order
    {
        lines = List.copyOf(lines);
    }
}
```

RESULT / BEHAVIOR

Mutating the caller’s original list no longer changes Order.lines().

COMMON PITFALL

Assuming a record deep-copies every reference component automatically.

APPLY THE RULE — QUESTION & ANSWER

Question: Is record.equals reference identity? Answer: No; generated equals compares component values.

REAL-WORLD RELEVANCE

DTOs, event payloads and small immutable data carriers often fit records.

Lesson 17 — Compact record constructors and validation

CORE RULE

A compact constructor runs validation and normalization before the compiler assigns record components.

WHY IT MATTERS

Use it to reject invalid values. Assignments to constructor parameters can normalize values; avoid hiding side effects or long-running logic in value construction.

JAVA EXAMPLE — ALLMAN STYLE

```
record Person(String name, int age)
{
    public Person
    {
        if (age < 0)
        {
            throw new IllegalArgumentException("age");
        }
    }
}
```

EXPECTED OUTPUT / BEHAVIOR

new Person("Ana", -1) throws IllegalArgumentException.

EXECUTION TRACE

The constructor body checks parameters; invalid age stops creation; valid parameters are assigned to the final components.

COMMON PITFALL

A record cannot declare additional instance fields beyond its components.

APPLY THE RULE

How do you normalize a string name?

Answer: Reassign constructor parameter name, such as name = name.trim(), after null validation.

Lesson 17 — Compact constructors and representation invariants

CONCEPTUAL EXECUTION MODEL

record constructor validation → normalize constructor parameters → compiler assigns record fields after compact constructor body.

DEEPER EXPLANATION / WHY IT WORKS

A compact record constructor names no parameter list; parameters correspond to record components. Validate required constraints and normalize input before the implicit field assignment. Defensive copies protect collection components; a mutable array component additionally needs accessor defensive copying. Ensure normalized values preserve equals/hashCode semantics.

JAVA EXAMPLE / ALLMAN BRACES

```
record Email(String address)
{
    Email
    {
        address = address.trim().toLowerCase(Locale.ROOT);
        if (address.isEmpty())
        {
            throw new IllegalArgumentException("address");
        }
    }
}
```

RESULT / BEHAVIOR

new Email(" A@B.COM ").address() returns a@b.com.

COMMON PITFALL

Using a compact constructor but leaving externally mutable component state unprotected.

APPLY THE RULE — QUESTION & ANSWER

Question: Can a record have ordinary instance fields beyond its components? Answer: No.

REAL-WORLD RELEVANCE

Business value objects can validate themselves at construction time.

Lesson 18 — Lombok code generation

CORE RULE

Lombok can transform the compiler-visible class using annotations such as @Getter, @Builder and @RequiredArgsConstructor.

WHY IT MATTERS

The methods may not appear in handwritten source but are visible to compiled code and often to IDE assistance. Its build configuration and plugin support matter. Lombok is an external dependency, unlike records.

JAVA EXAMPLE — ALLMAN STYLE

```
@Getter
@RequiredArgsConstructor
class User
{
    private final String name;
}
// Conceptual generated API: User(String name), getName()
```

EXPECTED OUTPUT / BEHAVIOR

No output until a caller invokes generated members.

EXECUTION TRACE

An annotation triggers Lombok’s compiler integration; accessors/constructors become available; callers invoke them like ordinary methods.

COMMON PITFALL

@Data on mutable entities may generate equals/hashCode with surprising behavior; inspect generated code.

APPLY THE RULE

Can Lombok work without being on the build classpath?

Answer: Not normally; its annotation processing integration must be configured.

Lesson 18 — Lombok compilation and IDE behavior

CONCEPTUAL EXECUTION MODEL

Lombok annotations → javac/compiler integration → generated methods in bytecode; editor plugins may need to recognize them.

DEEPER EXPLANATION / WHY IT WORKS

Lombok reduces boilerplate, but @Data-generated equals and toString deserve design review on entities with circular relationships or sensitive fields. @Builder is convenient but can bypass constructors or defaults unless carefully configured. Lombok is a project dependency; plain javac without it will not compile Lombok-based source. Keep examples runnable without Lombok unless that dependency is explicitly added.

JAVA EXAMPLE / ALLMAN BRACES

```
@Getter
@RequiredArgsConstructor
class Account
{
    private final String id;
}
```

RESULT / BEHAVIOR

With Lombok configured, constructor Account(String) and getter getId() are generated.

COMMON PITFALL

Blindly applying @Data to all JPA entities and assuming generated methods are harmless.

APPLY THE RULE — QUESTION & ANSWER

Question: Why can IDE and javac disagree? Answer: Plugin or annotation-processing configuration differs.

REAL-WORLD RELEVANCE

When reading a Spring application, inspect generated/compiled members instead of assuming they do not exist.

Lesson 19 — Annotation processors and generated sources

CORE RULE

A processor discovers annotated program elements at compile time and can emit new Java source or resources.

WHY IT MATTERS

Examples include MapStruct mappers, some DI tooling, configuration metadata and serialization adapters. Generated source directories depend on the build tool; rebuild before diagnosing missing generated classes.

JAVA EXAMPLE — ALLMAN STYLE

```
@Mapper
interface UserMapper
{
    UserDto toDto(User source);
}
// MapStruct may generate UserMapperImpl.
```

EXPECTED OUTPUT / BEHAVIOR

No output; generated source appears during configured compilation.

EXECUTION TRACE

javac invokes registered processors in rounds; the mapper processor emits an implementation; a later compilation round compiles generated source.

COMMON PITFALL

@Mapper by itself is meaningless if its processor dependency is missing.

APPLY THE RULE

Why might UserMapperImpl be absent?

Answer: Annotation processing may be disabled or not configured.

Lesson 19 — Annotation processor rounds and Filer output

CONCEPTUAL EXECUTION MODEL

annotated Element → processor round → Filer.createSourceFile → generated .java → subsequent javac compilation.

DEEPER EXPLANATION / WHY IT WORKS

An actual processor implements process(Set, RoundEnvironment). It must avoid recreating the same source in later rounds, handle IOException, and report diagnostics via Messager. The compiler discovers processor classes using service registration or -processor. For clean reproduction, keep processor and target demo in separate compilation stages.

JAVA EXAMPLE / ALLMAN BRACES

```
JavaFileObject file = processingEnv.getFiler()
    .createSourceFile("demo.GeneratedHello");
try (Writer w = file.openWriter())
{
    w.write("package demo; public class GeneratedHello {} ");
}
```

RESULT / BEHAVIOR

The compiler writes GeneratedHello.java into the generated-source output location.

COMMON PITFALL

Recreating an already generated file during a later processing round (FilerException).

APPLY THE RULE — QUESTION & ANSWER

Question: Is Filer a runtime reflection utility? Answer: No; it is a compiler annotation-processing API.

REAL-WORLD RELEVANCE

Mapping frameworks and schema/code generators often use similar compile-time processing.

Lesson 20 — Compile-time generation versus runtime proxy

CORE RULE

Generated sources are typically available as .java files during builds; runtime proxies can instead create behavior dynamically while an application runs.

WHY IT MATTERS

Spring AOP can wrap method calls at runtime, JPA providers may use proxies for lazy loading, and bytecode enhancement can modify classes. These are different mechanisms and should not all be called annotation processing.

JAVA EXAMPLE — ALLMAN STYLE

```
interface Service
{
    String greet(String name);
}
// Proxy wraps calls to Service at runtime.
```

EXPECTED OUTPUT / BEHAVIOR

Output depends on the invocation handler.

EXECUTION TRACE

Code declares an interface; a proxy factory creates an implementation dynamically; the invocation handler receives later method calls.

COMMON PITFALL

A runtime proxy does not necessarily produce a readable source file.

APPLY THE RULE

How does a JDK dynamic proxy choose its surface?

Answer: It implements supplied interfaces and routes calls through an InvocationHandler.

Lesson 20 — Compile-time code generation versus runtime proxy

CONCEPTUAL EXECUTION MODEL

Source generator → new Java source/bytecode at build time; runtime proxy → interception object while application runs.

DEEPER EXPLANATION / WHY IT WORKS

Compile-time processing can generate checked source artifacts that your compiler can validate. A JDK dynamic proxy generates an object implementing selected interfaces at runtime and routes calls through InvocationHandler. Bytecode-subclass proxies used by some frameworks have different limits involving final classes/methods and constructors. Be precise about which approach a given tool uses.

JAVA EXAMPLE / ALLMAN BRACES

```
Service s = (Service) Proxy.newProxyInstance(
    Service.class.getClassLoader(),
    new Class<?>[] {Service.class}, handler);
```

RESULT / BEHAVIOR

The runtime object implements Service and routes method calls to handler.

COMMON PITFALL

Treating all generated methods as source files or all proxies as interface-only mechanisms.

APPLY THE RULE — QUESTION & ANSWER

Question: Does JDK Proxy subclass your concrete class? Answer: No; it implements interfaces.

REAL-WORLD RELEVANCE

Helps explain why some Spring methods execute through advice and others do not.

Lesson 21 — Dynamic proxy pattern

CORE RULE

java.lang.reflect.Proxy can create an interface implementation whose invocation handler intercepts calls.

WHY IT MATTERS

A handler can add logging, validation and delegation. Object methods and checked exceptions deserve careful treatment in production; avoid implementing security rules solely through naive interception.

JAVA EXAMPLE — ALLMAN STYLE

```
Greeting g = (Greeting) Proxy.newProxyInstance(
    Greeting.class.getClassLoader(),
    new Class<?>[] {Greeting.class},
    (proxy, method, args) -> "Hello " + args[0]);
System.out.println(g.hello("Java"));
```

EXPECTED OUTPUT / BEHAVIOR

```
Hello Java
```

EXECUTION TRACE

The proxy implements Greeting; calling hello enters the handler; handler constructs the text and returns it to the caller.

COMMON PITFALL

JDK proxies need interfaces; subclass-based proxies use different mechanisms.

APPLY THE RULE

Can a JDK proxy directly subclass a concrete class?

Answer: No; it implements interfaces.

Lesson 21 — Dynamic proxy invocation and Object methods

CONCEPTUAL EXECUTION MODEL

JDK Proxy → InvocationHandler.invoke(proxy,method,args) for interface calls; define a careful strategy for Object methods.

DEEPER EXPLANATION / WHY IT WORKS

InvocationHandler receives a Method describing the invoked interface method. You may call the backing object reflectively, but unwrap InvocationTargetException and throw its cause. Interfaces determine the cast type of the returned proxy. Consider equals, hashCode and toString behavior explicitly if handlers need special behavior; otherwise surprising recursion or identity semantics can occur.

JAVA EXAMPLE / ALLMAN BRACES

```
InvocationHandler h = (proxy, method, args) ->
{
    System.out.println("before " + method.getName());
    return method.invoke(target, args);
};
```

RESULT / BEHAVIOR

A proxy service call prints before then executes the target method.

COMMON PITFALL

Accidentally calling method.invoke(proxy,args) inside its own handler, causing recursion.

APPLY THE RULE — QUESTION & ANSWER

Question: Where should the handler dispatch? Answer: Usually to the backing target, not the proxy itself.

REAL-WORLD RELEVANCE

Typical mechanism for timing, tracing or cross-cutting interception via interfaces.

Lesson 22 — Inspect generated behavior in IntelliJ

CORE RULE

When source seems to be missing a method, check navigation, generated sources, class files, build logs and framework proxy layers.

WHY IT MATTERS

Follow call sites with Find Usages; inspect the actual implementation or interface and check pom.xml for Lombok and processor dependencies. If a breakpoint skips a source method, consider generated bytecode or a proxy boundary.

JAVA EXAMPLE — ALLMAN STYLE

```
record Invoice(String id, int amount)
{
}
// In IntelliJ inspect generated members and compiled class.
```

EXPECTED OUTPUT / BEHAVIOR

No output; this is a code-navigation workflow.

EXECUTION TRACE

Find the declaration; inspect generated accessors; check the build pipeline; debug the actual runtime class when behavior differs.

COMMON PITFALL

Do not edit target/generated-sources as the source of truth.

APPLY THE RULE

What does a $$SpringCGLIB$$ name suggest?

Answer: A generated runtime subclass/proxy used by a framework.

Lesson 22 — Finding generated behavior in IntelliJ

CONCEPTUAL EXECUTION MODEL

call site → go to declaration → search generated sources → decompile class → inspect build or runtime proxy.

DEEPER EXPLANATION / WHY IT WORKS

IDE navigation may reveal a method that is not handwritten: record component accessor, Lombok-generated getter, MapStruct mapper, or runtime proxy invocation. For compilation-generated code, look under target/generated-sources/annotations or build/generated/sources. For proxy behavior, inspect stack traces, runtime class names and framework configuration; a proxy class might not exist in source.

JAVA EXAMPLE / ALLMAN BRACES

```
System.out.println(service.getClass().getName());
```

RESULT / BEHAVIOR

Proxy-backed services often have runtime class names different from their declared type.

COMMON PITFALL

Assuming an unresolved method must be missing source rather than produced by a compiler or framework.

APPLY THE RULE — QUESTION & ANSWER

Question: Can you always find a runtime proxy as a .java source file? Answer: No.

REAL-WORLD RELEVANCE

A repeatable tracing workflow is essential when entering an unfamiliar enterprise repository.

Lesson 23 — Framework annotation flow and tradeoffs

CORE RULE

Trace annotations to the component that interprets them, not merely to their definitions.

WHY IT MATTERS

In Spring, a component scanner, bean factory, transaction interceptor, validation provider or web handler mapping may interpret different annotations at different points. Distinguish configuration metadata, compile-time generation and runtime interception.

JAVA EXAMPLE — ALLMAN STYLE

```
@Service
class BillingService
{
    @Transactional
    public void charge()
    {
    }
}
```

EXPECTED OUTPUT / BEHAVIOR

No direct console output; behavior requires Spring runtime.

EXECUTION TRACE

Bean scanning discovers the service; container creates a bean; transactional advice may wrap eligible calls through proxy interception.

COMMON PITFALL

Self-invocation can bypass Spring proxy advice in common proxy-based arrangements.

APPLY THE RULE

Would @Transactional do anything in a plain Java main without Spring?

Answer: No; an appropriate framework infrastructure must interpret it.

Lesson 23 — Framework annotation handling and constraints

CONCEPTUAL EXECUTION MODEL

annotation metadata → framework scanner → registered bean/handler → runtime advice or binding → actual service call.

DEEPER EXPLANATION / WHY IT WORKS

An annotation can be read at build time or runtime, and different frameworks implement very different behaviors. For example, Spring transactional advice commonly relies on proxy interception. Calls from a target object to its own methods can bypass proxy advice in common proxy-based configurations; final/private methods can have constraints. These are framework concerns, not intrinsic behavior of @interface.

JAVA EXAMPLE / ALLMAN BRACES

```
interface BillingService
{
    void charge();
}
// Framework proxy may intercept charge().
```

RESULT / BEHAVIOR

Whether advice runs depends on framework wiring and the call path.

COMMON PITFALL

Assuming an annotated method is intercepted even when the call never passes through its proxy.

APPLY THE RULE — QUESTION & ANSWER

Question: What must you trace? Answer: Who creates/wires the proxy and whether invocation reaches it.

REAL-WORLD RELEVANCE

Crosses directly into Chapter 17 dependency injection and transactions.

Lesson 24 — Integrated feature walkthrough

CORE RULE

Combine custom metadata, runtime scanning, reflection and generated business classes without confusing their roles.

WHY IT MATTERS

A complete feature may define an annotation, scan methods for it, invoke annotated handlers, use records as input/output DTOs and rely on generated mappers. Keep discovery and dispatch in a dedicated infrastructure component rather than in core domain logic.

JAVA EXAMPLE — ALLMAN STYLE

```
@Retention(RetentionPolicy.RUNTIME)
@interface Command
{
    String value();
}
@Command("submit")
void submit()
{
    System.out.println("submitted");
}
```

EXPECTED OUTPUT / BEHAVIOR

submitted (only when a dispatcher invokes submit).

EXECUTION TRACE

Annotation survives compilation; dispatcher discovers matching method; invocation executes its business body; normal output appears.

COMMON PITFALL

Scan a bounded set of known classes, validate invocable signatures and preserve invocation causes.

APPLY THE RULE

What changes if retention is CLASS?

Answer: Ordinary runtime reflection no longer sees the annotation.

Lesson 24 — Integrated trace from metadata to behavior

CONCEPTUAL EXECUTION MODEL

declare @Handler → scan method → validate signature → register → invoke request via reflection → diagnose target exception.

DEEPER EXPLANATION / WHY IT WORKS

A realistic scanner must reject duplicate keys, verify public no-arg or agreed signatures, provide deterministic diagnostics, and preserve the called method’s exception cause. Security requires a whitelist of scan targets. A processor-based alternative can validate metadata at compile time and generate dispatch code, reducing runtime lookup but adding build-system complexity.

JAVA EXAMPLE / ALLMAN BRACES

```
for (Method m : Service.class.getDeclaredMethods())
{
    Handler a = m.getAnnotation(Handler.class);
    if (a != null)
    {
        // Validate signature; register handler by a.value().
    }
}
```

RESULT / BEHAVIOR

The registry contains only explicitly annotated, valid methods.

COMMON PITFALL

Scanning arbitrary packages and invoking methods based directly on untrusted request strings.

APPLY THE RULE — QUESTION & ANSWER

Question: How do you handle a target failure? Answer: Unwrap InvocationTargetException and preserve the cause.

REAL-WORLD RELEVANCE

End-to-end picture: custom annotation, scanner, reflection, optional proxy interception and compiled alternatives.

Chapter 9 — Consolidated Reference

Annotations are metadata. Retention determines discoverability, target determines legal placement, and processors/frameworks determine behavior. Reflection discovers types and members at runtime, while access control and module rules still apply. Records generate component-based value behavior; Lombok and annotation processors produce compiler-visible code; proxies implement call interception at runtime.

Debugging route

Start from a call site → inspect declared class and interfaces → find annotations → determine who reads each annotation → check build processors → check generated source or compiled classes → inspect the runtime type for proxies → follow failure causes.

Read next

Chapter 10: Files, Configuration and JSON. Keep this chapter as a reference when Spring behavior seems to appear without handwritten method calls.